Red Hat Quay 3.18

Securing Red Hat Quay

Securing Red Hat Quay

Last Updated: 2026-07-29

Red Hat Quay 3.18 Securing Red Hat Quay

Securing Red Hat Quay

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Securing Red Hat Quay: SSL/TLS, Certificates, and Encryption

Table of Contents

PREFACE

Red Hat Quay offers administrators the ability to secure communication and trusted access to their repositories through the use of Transport Layer Security (TLS), certificate management, and encryption techniques. Properly configuring SSL/TLS and implementing custom certificates can help safeguard data, secure external connections, and maintain trust between Red Hat Quay and the integrated services of your choosing.

The following topics are covered:

- Configuring custom SSL/TLS certificates for standalone Red Hat Quay deployments
- Configuring custom SSL/TLS certificates for Red Hat Quay on OpenShift Container Platform
- Inheriting the OpenShift Container Platform cluster TLS security profile for Red Hat Quay on OpenShift Container Platform
- Referencing an external TLS Secret for automated certificate rotation on Red Hat Quay on OpenShift Container Platform
- Enabling TLS encryption for Operator-managed PostgreSQL on Red Hat Quay on OpenShift Container Platform
- Adding additional Certificate Authorities to the Red Hat Quay container
- Adding additional Certificate Authorities to Red Hat Quay on OpenShift Container Platform

CHAPTER 1. CONFIGURING SSL AND TLS FOR RED HAT QUAY

Transport Layer Security (TLS) is a cryptographic protocol that secures network communication between clients and the Red Hat Quay server. You can configure Red Hat Quay with certificates from a trusted Certificate Authority or with self-signed certificates for internal use.

1.1. CREATING A CERTIFICATE AUTHORITY

To secure your Red Hat Quay deployment with self-signed certificates, you can create a root certificate authority and generate a server certificate for your registry hostname. You can use OpenSSL to create the CA key, signing request, and certificate files.

Procedure

1. Generate the root CA key by entering the following command:

```
$ openssl genrsa -out rootCA.key 2048
```

2. Generate the root CA certificate by entering the following command:

```
$ openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 1024 -out rootCA.pem
```

3. Enter the information to incorporate into your certificate request, including the server hostname, for example:

```
Country Name (2 letter code) [XX]:IE
State or Province Name (full name) []:GALWAY
Locality Name (eg, city) [Default City]:GALWAY
Organization Name (eg, company) [Default Company Ltd]:QUAY
Organizational Unit Name (eg, section) []:DOCS
Common Name (eg, your name or your server's hostname) []:quay-server.example.com
```

4. Generate the server key by entering the following command:

```
$ openssl genrsa -out ssl.key 2048
```

5. Generate a signing request by entering the following command:

```
$ openssl req -new -key ssl.key -out ssl.csr
```

6. Enter the information to incorporate into your certificate request, including the server hostname, for example:

```
Country Name (2 letter code) [XX]:IE
State or Province Name (full name) []:GALWAY
Locality Name (eg, city) [Default City]:GALWAY
Organization Name (eg, company) [Default Company Ltd]:QUAY
Organizational Unit Name (eg, section) []:DOCS
Common Name (eg, your name or your server's hostname) []:quay-server.example.com
Email Address []:
```

7. Create a configuration file **openssl.cnf**, specifying the server hostname, for example:

Example openssl.cnf file

```
[req]
req_extensions = v3_req
distinguished_name = req_distinguished_name
[req_distinguished_name]
[ v3_req ]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names
[alt_names]
DNS.1 = <quay-server.example.com>
IP.1 = 192.168.1.112
```

8. Use the configuration file to generate the certificate **ssl.cert**:

```
$ openssl x509 -req -in ssl.csr -CA rootCA.pem -CAkey rootCA.key -CAcreateserial -out
ssl.cert -days 356 -extensions v3_req -extfile openssl.cnf
```

9. Confirm your created certificates and files by entering the following command:

```
$ ls /path/to/certificates
```

Example output

```
rootCA.key ssl-bundle.cert ssl.key custom-ssl-config-bundle-secret.yaml rootCA.pem ssl.cert
openssl.cnf rootCA.srl  ssl.csr
```

1.2. CONFIGURING SSL/TLS FOR STANDALONE RED HAT QUAY DEPLOYMENTS

For standalone Red Hat Quay deployments, you configure SSL/TLS certificates by using the CLI and updating **config.yaml** manually. Child procedures in this section cover certificate creation, configuration, testing, and trust setup.

1.2.1. Configuring custom SSL/TLS certificates by using the command line interface

To enable custom SSL/TLS certificates on your Red Hat Quay deployment, you can copy certificate files to your configuration directory and update the **config.yaml** file to use HTTPS. You can then restart the registry container to apply the SSL/TLS configuration.

Prerequisites

- You have created a certificate authority and signed the certificate.

Procedure

1. Copy the certificate file and primary key file to your configuration directory, ensuring they are named **ssl.cert** and **ssl.key** respectively:

```
cp ~/ssl.cert ~/ssl.key /path/to/configuration_directory
```

2. Navigate to the configuration directory by entering the following command:

```
$ cd /path/to/configuration_directory
```

3. Edit the **config.yaml** file and specify that you want Red Hat Quay to handle SSL/TLS:

Example config.yaml file

```
# ...
SERVER_HOSTNAME: <quay-server.example.com>
...
PREFERRED_URL_SCHEME: https
# ...
```

4. Optional: Append the contents of the **rootCA.pem** file to the end of the **ssl.cert** file by entering the following command:

```
$ cat rootCA.pem >> ssl.cert
```

5. Stop the **Quay** container by entering the following command:

```
$ sudo podman stop <quay_container_name>
```

6. Restart the registry by entering the following command:

```
$ sudo podman run -d --rm -p 80:8080 -p 443:8443 \
  --name=quay \
  -v $QUAY/config:/conf/stack:Z \
  -v $QUAY/storage:/datastorage:Z \
  registry.redhat.io/quay/quay-rhel9:v3.18.0
```

1.2.2. Configuring Podman to trust the Certificate Authority

To configure Podman to trust your self-signed certificate authority for Red Hat Quay, you can copy the root CA to the hostname-specific certificate directory under **/etc/containers/certs.d/** or **/etc/docker/certs.d/**. You can verify the setup by logging in to your registry without the **--tls-verify=false** option.

Procedure

1. Copy the root CA file to one of **/etc/containers/certs.d/** or **/etc/docker/certs.d/**. Use the exact path determined by the server hostname, and name the file **ca.crt**:

```
$ sudo cp rootCA.pem /etc/containers/certs.d/quay-server.example.com/ca.crt
```

2. Verify that you no longer need to use the **--tls-verify=false** option when logging in to your Red Hat Quay registry:

```
$ sudo podman login quay-server.example.com
```

Example output

```
-
```

```
Login Succeeded!
```

1.2.3. Configuring the system to trust the certificate authority

To trust your self-signed certificate authority, you can add the root CA to the system-wide trust store and update certificate configuration. You can verify trust with the trust list command before browsing your Red Hat Quay registry over HTTPS.

Procedure

1. Enter the following command to copy the **rootCA.pem** file to the consolidated system-wide trust store:

```
$ sudo cp rootCA.pem /etc/pki/ca-trust/source/anchors/
```

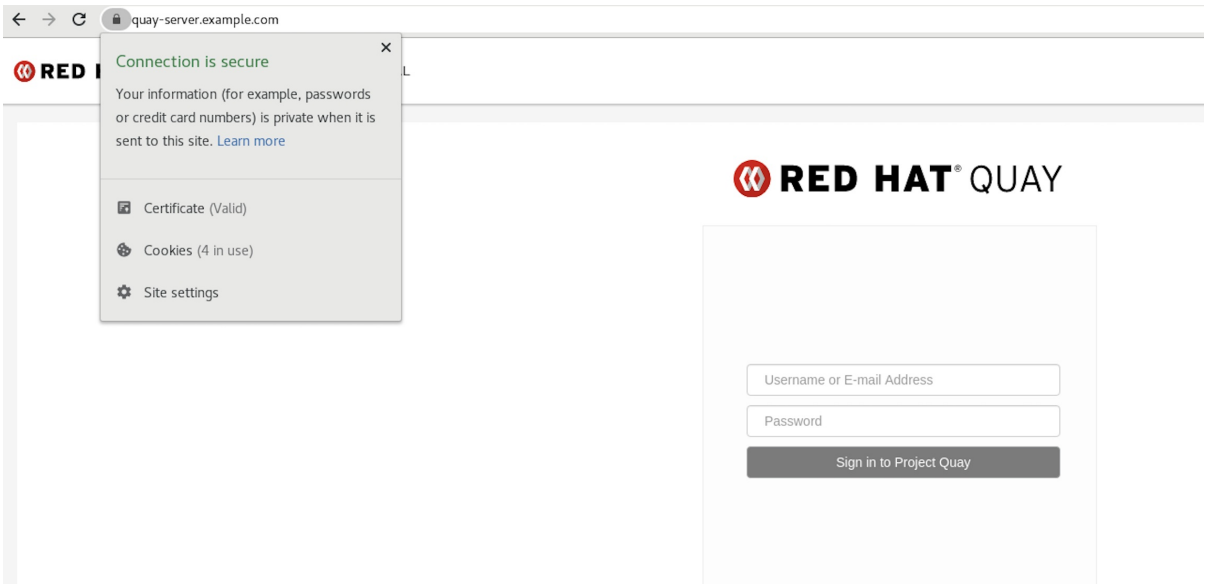
2. Enter the following command to update the system-wide trust store configuration:

```
$ sudo update-ca-trust extract
```

3. Optional. You can use the **trust list** command to ensure that the **Quay** server has been configured:

```
$ trust list | grep quay
    label: quay-server.example.com
```

Now, when you browse to the registry at **https://quay-server.example.com**, the lock icon shows that the connection is secure:



4. To remove the **rootCA.pem** file from system-wide trust, delete the file and update the configuration:

```
$ sudo rm /etc/pki/ca-trust/source/anchors/rootCA.pem
```

```
$ sudo update-ca-trust extract
```

```
$ trust list | grep quay
```

Additional resources

- Using shared system certificates

1.3. CONFIGURING CUSTOM SSL/TLS CERTIFICATES FOR RED HAT QUAY ON OPENSHIFT CONTAINER PLATFORM

When you deploy Red Hat Quay on OpenShift Container Platform, the Operator can use OpenShift Container Platform Certificate Authority certificates or custom SSL/TLS certificates in the config bundle. You can configure custom certificates before or after initial deployment by updating the configBundleSecret and setting the tls component to unmanaged.

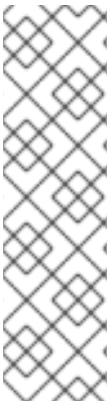
When Red Hat Quay is deployed on OpenShift Container Platform, the **tls** component of the **QuayRegistry** custom resource definition (CRD) is set to **managed** by default. As a result, OpenShift Container Platform's Certificate Authority is used to create HTTPS endpoints and to rotate SSL/TLS certificates.

You can configure custom SSL/TLS certificates before or after the initial deployment of Red Hat Quay on OpenShift Container Platform. This process involves creating or updating the **configBundleSecret** resource within the **QuayRegistry** YAML file to integrate your custom certificates and setting the **tls** component to **unmanaged**.

IMPORTANT

When configuring custom SSL/TLS certificates for Red Hat Quay, administrators are responsible for certificate rotation.

The following procedures enable you to apply custom SSL/TLS certificates to ensure secure communication and meet specific security requirements for your Red Hat Quay on OpenShift Container Platform deployment. These steps assumed you have already created a Certificate Authority (CA) bundle or an **ssl.key**, and an **ssl.cert**. The procedure then shows you how to integrate those files into your Red Hat Quay on OpenShift Container Platform deployment, which ensures that your registry operates with the specified security settings and conforms to your organization's SSL/TLS policies.



NOTE

- The following procedure is used for securing Red Hat Quay with an HTTPS certificate. Note that this differs from managing Certificate Authority Trust Bundles. CA Trust Bundles are used by system processes within the **Quay** container to verify certificates against trusted CAs, and ensure that services like LDAP, storage backend, and OIDC connections are trusted.
- If you are adding the certificates to an existing deployment, you must include the existing **config.yaml** file in the new config bundle secret, even if you are not making any configuration changes.

Additional resources

- Referencing an external TLS Secret for Red Hat Quay on OpenShift Container Platform

1.4. OPENSHIFT CONTAINER PLATFORM CLUSTER TLS SECURITY PROFILE INHERITANCE

When you deploy Red Hat Quay on OpenShift Container Platform, the Red Hat Quay Operator can read the cluster-wide **tlsSecurityProfile** from the OpenShift Container Platform **APIServer** cluster resource and apply the corresponding **SSL_PROTOCOLS** and **SSL_CIPHERS** settings to the registry and mirror workers. This aligns Red Hat Quay with the platform TLS policy without manual configuration.

1.4.1. How TLS profile inheritance works

The Operator reads **spec.tlsSecurityProfile** from the **APIServer** cluster resource and translates the profile into Red Hat Quay **SSL_PROTOCOLS** and **SSL_CIPHERS** values in the generated **config.yaml**. Supported profile types are **Old**, **Intermediate**, **Modern**, and **Custom**. If the cluster has no profile set, the Operator defaults to the **Intermediate** profile (TLS 1.2 and TLS 1.3).

To fully override cluster-profile inheritance, set both **SSL_PROTOCOLS** and **SSL_CIPHERS** in the **configBundleSecret**. Setting either field disables inheritance for both fields. On Kubernetes clusters without the **config.openshift.io** API, the Operator does not inject TLS settings and Red Hat Quay uses its built-in defaults.

1.4.2. Managed TLS and unmanaged TLS

How the cluster TLS security profile applies depends on whether the Operator or Red Hat Quay terminates TLS:

- When the **tls** component is set to **managed**, the OpenShift Container Platform Route enforces the cluster TLS profile. No additional TLS protocol or cipher configuration is required in the config bundle.
- When the **tls** component is set to **unmanaged**, Red Hat Quay terminates TLS directly. When neither **SSL_PROTOCOLS** nor **SSL_CIPHERS** is set in the config bundle, Red Hat Quay inherits the cluster TLS security profile.

Additional resources

- Configuring custom SSL/TLS certificates for Red Hat Quay on OpenShift Container Platform

1.5. PRESERVING TLS SETTINGS BEFORE UPGRADING RED HAT QUAY ON OPENSHIFT CONTAINER PLATFORM

Starting with Red Hat Quay 3.18, the Red Hat Quay Operator inherits the cluster-wide TLS security profile from the OpenShift **APIServer** configuration when neither **SSL_PROTOCOLS** nor **SSL_CIPHERS** is set in the **configBundleSecret** resource. Before upgrading to 3.18, review your cluster profile and, if needed, set both fields explicitly to preserve your current TLS behavior.

When the **tls** component is set to **managed**, the OpenShift Route already enforces the cluster TLS profile and no action is required. When the **tls** component is set to **unmanaged**, Red Hat Quay terminates TLS directly and inherits the cluster profile after upgrade unless you override it in the config bundle. If you use unmanaged TLS and have already set both **SSL_PROTOCOLS** and **SSL_CIPHERS** in the config bundle, no additional TLS configuration is required before upgrading. Setting only one of these fields disables cluster-profile inheritance for both fields.

Prerequisites

- You have cluster administrator access to review the OpenShift Container Platform **APIServer** configuration.
- You can edit the **configBundleSecret** referenced by your **QuayRegistry** custom resource (CR).

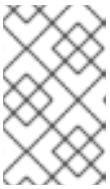
Procedure

1. Review the cluster TLS security profile:

```
$ oc get apiserver cluster -o jsonpath='{.spec.tlsSecurityProfile}{"\n"}'
```

2. If you must preserve your current TLS settings, add **SSL_PROTOCOLS** and **SSL_CIPHERS** to the **config.yaml** file in your **configBundleSecret** before upgrading. For example:

```
# ...
SSL_PROTOCOLS:
  - TLSv1.2
  - TLSv1.3
SSL_CIPHERS:
  - ECDHE-RSA-AES128-GCM-SHA256
  - ECDHE-ECDSA-AES128-GCM-SHA256
  - ECDHE-RSA-AES256-GCM-SHA384
  - ECDHE-ECDSA-AES256-GCM-SHA384
  # Add other required ciphers
# ...
```



NOTE

Include every cipher suite your clients require. To fully override cluster-profile inheritance, set both **SSL_PROTOCOLS** and **SSL_CIPHERS**. Setting either field disables inheritance for both fields.

3. Update the **configBundleSecret** with the modified **config.yaml** file. You can edit the secret in the OpenShift Container Platform web console or recreate it from a local file. For example:

```
$ oc create secret generic <config_bundle_secret_name> \
  --from-file config.yaml=./config.yaml \
  --dry-run=client -o yaml | oc apply -f -
```

For more information, see Modifying the configuration file by using the OpenShift Container Platform web console and Overriding the cluster TLS security profile .

4. Proceed with the Red Hat Quay Operator upgrade. If the cluster TLS profile is acceptable and neither **SSL_PROTOCOLS** nor **SSL_CIPHERS** is set, no additional TLS configuration is required.

Additional resources

- SSL/TLS configuration fields
- OpenShift Container Platform cluster TLS security profile inheritance

1.5.1. Creating a custom SSL/TLS configBundleSecret resource

To upload custom SSL/TLS certificates to Red Hat Quay on OpenShift Container Platform, you can create a **configBundleSecret** resource that includes your **ssl.cert** and **ssl.key files** and reference it from the **QuayRegistry** custom resource. You then set the tls component to unmanaged so Red Hat Quay terminates TLS with your certificates.

Prerequisites

- You have base64 decoded the original config bundle into a **config.yaml** file. For more information, see Downloading the existing configuration.
- You have generated custom SSL certificates and keys.

Procedure

1. Create a new YAML file, for example, **custom-ssl-config-bundle-secret.yaml**:

```
$ touch custom-ssl-config-bundle-secret.yaml
```

2. Create the **custom-ssl-config-bundle-secret** resource.

   a. Create the resource by entering the following command:

```
$ oc -n <namespace> create secret generic custom-ssl-config-bundle-secret \
  --from-file=config.yaml=</path/to/config.yaml> \
  --from-file=ssl.cert=</path/to/ssl.cert> \
  --from-file=extra_ca_cert_<name-of-certificate>.crt=ca-certificate-bundle.crt \
  --from-file=ssl.key=</path/to/ssl.key> \
  --dry-run=client -o yaml > custom-ssl-config-bundle-secret.yaml
```

   where:

   --from-file=config.yaml=</path/to/config.yaml>

   Specifies your base64 decoded **config.yaml** file.

   --from-file=ssl.cert=</path/to/ssl.cert>

   Specifies your **ssl.cert** file.

   --from-file=extra_ca_cert_<name-of-certificate>.crt=ca-certificate-bundle.crt

   Specifies an additional CA PEM using a Secret key that starts with **extra_ca_cert_** (for example, **--from-file=extra_ca_cert_<name-of-certificate>.crt=ca-certificate-bundle.crt**). The Red Hat Quay Operator writes these files under **conf/stack/extra_ca_certs/** in the deployed bundle. For LDAP, OIDC, or other integrations that need custom CAs, supply the PEMs this way and use **conf/stack/extra_ca_certs/<file>** in **config.yaml** when a field such as **ssl_ca_path** requires an explicit path. This parameter is optional.

   --from-file=ssl.key=</path/to/ssl.key>

   Specifies your **ssl.key** file.

3. Optional. You can check the content of the **custom-ssl-config-bundle-secret.yaml** file by entering the following command:

```
$ cat custom-ssl-config-bundle-secret.yaml
```

-

Example output

```
apiVersion: v1
data:
  config.yaml:
QUxMT1dfUFVMTFNfV0lUSE9VVF9TVFJJQ1RfTE9HR0lORzogZmFsc2UKQVVUSEVOVElD
QVRJT05fVFlQRTogRGF0YWJhc2UKREVGQVVMVF9UQUdfRVhQSVJBVElPTjogMncKREl
TVFJJQlVURURfU1R...
  ssl.cert:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUVYakNDQTBhZ0F3SUJBZ0lVTUFBR
k1YVWlWVHNoMGxNTWI3U1l0eFV5eTJjd0RRWUpLb1pJaHZjTkFRUwKQlFBd2dZZ3hDek
FKQmdOVkJBWVR...
  extra_ca_cert_<name-of-
certificate>:LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUVYakNDQTBhZ0F3SUJB
Z0lVTUFBRk1YVWlWVHNoMGxNTWI3U1l0eFV5eTJjd0RRWUpLb1pJaHZjTkFRUwKQlFBd
2dZZ3hDe...
  ssl.key:
LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0tCk1JSUV2UUlCQURBTkJna3Foa2lHOXcwQk
FRRUZBQVNDQktjd2dnU2pBZ0VBQW9JQkFRQ2c0VWxZOVV1SVJPY1oKcFhpZk9MVEdqa
S9neUxQMlpiMXQ...
kind: Secret
metadata:
  creationTimestamp: null
  name: custom-ssl-config-bundle-secret
  namespace: <namespace>
```

4. Create the **configBundleSecret** resource by entering the following command:

```
$ oc create -n <namespace> -f custom-ssl-config-bundle-secret.yaml
```

Example output

```
secret/custom-ssl-config-bundle-secret created
```

5. Update the **QuayRegistry** YAML file to reference the **custom-ssl-config-bundle-secret** object by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"configBundleSecret":"custom-ssl-config-bundle-secret"}}'
```

Example output

```
quayregistry.quay.redhat.com/example-registry patched
```

6. Set the **tls** component of the **QuayRegistry** YAML to **False** by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"components":[{"kind":"tls","managed":false}]}}'
```

Example output

-

```
quayregistry.quay.redhat.com/example-registry patched
```

7. Ensure that your **QuayRegistry** YAML file has been updated to use the custom SSL **configBundleSecret** resource, and that your and **tls** resource is set to **False** by entering the following command:

```
$ oc get quayregistry <registry_name> -n <namespace> -o yaml
```

Example output

```
# ...
  configBundleSecret: custom-ssl-config-bundle-secret
# ...
spec:
  components:
  - kind: tls
    managed: false
# ...
```

Verification

- Confirm a TLS connection to the server and port by entering the following command:

```
$  openssl s_client -connect <quay-server.example.com>:443
```

Example output

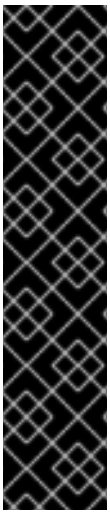
```
# ...
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
    Session-ID:
0E995850DC3A8EB1A838E2FF06CE56DBA81BD8443E7FA05895FBD6FBDE9FE737
    Session-ID-ctx:
    Resumption PSK:
1EA68F33C65A0F0FA2655BF9C1FE906152C6E3FEEE3AEB6B1B99BA7C41F06077989352
C58E07CD2FBDC363FA8A542975
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 7200 (seconds)

# ...
```

1.5.2. Referencing an external TLS Secret for Red Hat Quay on OpenShift Container Platform

You can reference an external **kubernetes.io/tls** Secret from the **tls** component of the **QuayRegistry** custom resource (CR). The Red Hat Quay Operator uses the certificate and private key from that Secret instead of embedding **ssl.cert** and **ssl.key** files in the **configBundleSecret** resource. This approach supports automated certificate rotation from sources such as cert-manager, HashiCorp Vault, or manual Secret updates.

When you use an external TLS Secret, set **spec.components[kind: tls].managed** to **false** and specify **secretRef**. The Operator watches the referenced Secret and performs a rolling restart of Red Hat Quay pods when the certificate data changes.



IMPORTANT

- **secretRef** is valid only when the **tls** component is unmanaged (**managed: false**).
- Do not configure both **secretRef** and **ssl.cert** / **ssl.key** files in the **configBundleSecret** resource. The Operator reports a conflict if both TLS sources are present.
- The certificate Subject Alternative Name (SAN) must include the hostname clients use to reach the registry. If the hostname does not match, the Operator can report **RolloutBlocked=True** until the certificate is corrected.
- Existing deployments that use embedded **ssl.cert** and **ssl.key** files in the config bundle continue to work without changes.

Prerequisites

- You have deployed the Red Hat Quay Operator and a **QuayRegistry** CR.
- You have a TLS Secret of type **kubernetes.io/tls** in the same namespace as the **QuayRegistry**, with **tls.crt** and **tls.key** data keys.
- The certificate and private key are valid (format, key match, chain, and hostname).

Procedure

1. Create a TLS Secret in the registry namespace, for example:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-quay-tls
  namespace: <namespace>
type: kubernetes.io/tls
data:
  tls.crt: <base64_encoded_certificate>
  tls.key: <base64_encoded_private_key>
```

2. Set the **tls** component to unmanaged and reference the Secret by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"components":[{"kind":"tls","managed":false,"secretRef":{"name":"my-quay-tls"}}]}}'
```

3. Verify that the **QuayRegistry** CR contains the expected configuration:

```
$ oc get quayregistry <registry_name> -n <namespace> -o yaml
```

Example output

-

```
spec:
  components:
  - kind: tls
    managed: false
    secretRef:
      name: my-quay-tls
```

4. Wait for the Operator to reconcile the registry. When certificate data in the referenced Secret changes, the Operator triggers a rolling restart of Red Hat Quay pods to load the updated certificate.

5. Verify that TLS from the external Secret is ready by entering the following command. When the TLS component is healthy, the **ComponentTLSReady** condition reports **status: "True"**.

```
$ oc wait quayregistry <registry_name> -n <namespace> --
for=condition=ComponentTLSReady=True --timeout=300s
```

You can also review all component conditions in the OpenShift Container Platform web console on the **QuayRegistry** details page, or by entering **oc get quayregistry <registry_name> -n <namespace> -o yaml** and checking **status.conditions**.

1.5.2.1. Using cert-manager with an external TLS Secret

You can install cert-manager on your cluster, issue a certificate into a **kubernetes.io/tls** Secret, and reference that Secret from the **tls** component of your **QuayRegistry** CR. When cert-manager renews the certificate, the Red Hat Quay Operator detects the updated Secret and rolls out updated pods.

Prerequisites

- You have installed cert-manager on the cluster. For more information, see the cert-manager documentation.
- You have a **ClusterIssuer** or **Issuer** that can issue certificates for your registry hostname.
- You know the hostname clients use to reach the registry. After deployment, this hostname is often available in **status.registryEndpoint** on the **QuayRegistry** CR.

Procedure

1. Create a **ClusterIssuer** CR, for example a self-signed issuer for testing:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
```

2. Create a **Certificate** CR that writes to the Secret referenced by your **QuayRegistry secretRef**. Set **dnsNames** to the registry route hostname, for example the host in **status.registryEndpoint**:

```
apiVersion: cert-manager.io/v1
kind: Certificate
```

```
metadata:
  name: quay-tls
  namespace: <namespace>
spec:
  secretName: quay-tls
  duration: 2160h
  renewBefore: 360h
  issuerRef:
    name: selfsigned-issuer
    kind: ClusterIssuer
  dnsNames:
  - <quay_route_hostname>
```

3. Configure the **QuayRegistry** CR to reference the Secret. For example:

```
apiVersion: quay.redhat.com/v1
kind: QuayRegistry
metadata:
  name: <registry_name>
  namespace: <namespace>
spec:
  components:
  - kind: tls
    managed: false
    secretRef:
      name: quay-tls
```

4. Wait for cert-manager to populate the TLS Secret. Then, confirm that the Operator has reconciled the registry by running the following command:

```
$ oc wait quayregistry <registry_name> -n <namespace> --
for=condition=ComponentTLSReady=True --timeout=300s
```

5. Optional. Force certificate renewal to verify that rotation triggers a rollout. This requires the cert-manager command-line plugin:

```
$ oc cert-manager renew quay-tls -n <namespace>
```

   After renewal, wait again for **ComponentTLSReady=True** and confirm that Red Hat Quay pods were replaced.

6. Optional. Delete the TLS Secret by running the following command:

```
$ oc delete secret quay-tls -n <namespace>
```

CHAPTER 2. CERTIFICATE-BASED AUTHENTICATION BETWEEN RED HAT QUAY AND SQL

You can configure certificate-based authentication between Red Hat Quay and SQL databases such as PostgreSQL and GCP CloudSQL by supplying client-side SSL/TLS certificates. This approach verifies the server certificate against a trusted Certificate Authority and supports automated deployments.

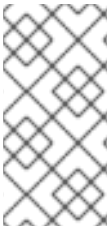
2.1. TLS ENCRYPTION FOR OPERATOR-MANAGED POSTGRESQL

You can enable TLS encryption for Operator-managed PostgreSQL databases used by Red Hat Quay and Clair. Configure TLS through the **overrides.tls** fields on the **postgres** and **clairpostgres** components in the **QuayRegistry** custom resource.

IMPORTANT

This feature configures transport encryption for Operator-managed PostgreSQL connections. It is distinct from the managed **tls** component, which handles Red Hat Quay's external HTTPS endpoint.

When TLS is enabled, the Operator configures PostgreSQL to accept encrypted connections and mounts the TLS certificates on the database pod. TLS is opt-in. Existing deployments continue to use unencrypted database connections until you explicitly enable it.



NOTE

After you enable TLS, confirm that PostgreSQL reports **ssl = on**. Optionally verify that Red Hat Quay or Clair client sessions appear in **pg_stat_ssl**, or that the registry database URI includes an **sslmode** setting such as **verify-full**, depending on your Operator version and configuration.

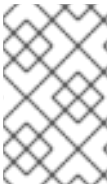
You can enable TLS independently for the **postgres** and **clairpostgres** components.

2.1.1. Certificate options

Table 2.1. Operator-managed PostgreSQL TLS certificate sources

Option	Description
OpenShift Container Platform Service CA (default on OpenShift Container Platform)	When you set **overrides.tls.enabled: true** without a **secretRef**, the Operator uses the OpenShift Container Platform Service CA on clusters that support Routes. The Operator annotates the PostgreSQL Service with **service.beta.openshift.io/serving-cert-secret-name** and mounts the generated serving certificate.
Operator-generated self-signed certificates	On Kubernetes clusters without the OpenShift Container Platform Service CA, the Operator generates ECDSA P-256 certificates with a 10-year validity period when no **secretRef** is provided.

Option	Description
User-provided or cert-manager certificates	Reference a Secret containing **ca.crt**, **tls.crt**, and **tls.key** by using **overrides.tls.secretRef.name**. This format is compatible with cert-manager output. When the referenced Secret changes, the Operator reconciles the deployment.



NOTE

This feature does not provide mutual TLS (mTLS) or automatic certificate rotation. To integrate with enterprise PKI or automate rotation, use **secretRef** with cert-manager or your own certificate management process.

Additional resources

- cert-manager Operator for Red Hat OpenShift

2.1.2. Enabling TLS for Operator-managed PostgreSQL

To encrypt database traffic for Operator-managed PostgreSQL used by Red Hat Quay and Clair, you can set **overrides.tls.enabled: true** on the **postgres** and **clairpostgres** components in the **QuayRegistry** custom resource.

Prerequisites

- The **postgres** and/or **clairpostgres** components that you want to protect are set to **managed: true**.
- You have access to edit the **QuayRegistry** custom resource in your registry namespace.

Procedure

1. Edit your **QuayRegistry** custom resource. For example:

```
$ oc edit quayregistry <registry_name> -n <namespace>
```

2. Under **spec.components**, add **overrides.tls.enabled: true** to the managed PostgreSQL components. For example:

```
spec:
  components:
    - kind: postgres
      managed: true
      overrides:
        tls:
          enabled: true
    - kind: clairpostgres
      managed: true
      overrides:
        tls:
          enabled: true
```

3. Save the changes and wait for the Operator to reconcile the registry.

NOTE

On OpenShift Container Platform, the Operator might briefly report **RolloutBlocked** while it waits for the Service CA to create the serving certificate Secret. Reconciliation retries until the Secret is available.

4. Verify that PostgreSQL has SSL enabled:

```
$ oc exec -n <namespace> deploy/<registry_name>-quay-database -c postgres -- \
  bash -lc 'psql -U "$POSTGRESQL_USER" -d "$POSTGRESQL_DATABASE" -tAc "SHOW
ssl;"'
```

The command should return **on**. For Clair, run the same check against the Clair PostgreSQL deployment when that component is managed.

5. Verify that the **QuayRegistry** status shows a healthy deployment and that **RolloutBlocked** is not stuck in an error state.

6. Optional: Confirm client TLS sessions with the following command:

```
$ oc exec -n <namespace> deploy/<registry_name>-quay-database -c postgres -- \
  bash -lc 'psql -U "$POSTGRESQL_USER" -d "$POSTGRESQL_DATABASE" -tAc
"SELECT count(*) FROM pg_stat_ssl s JOIN pg_stat_activity a ON s.pid = a.pid WHERE
s.ssl = true AND a.client_addr IS NOT NULL;"'
```

A count greater than **0** indicates active TLS client connections. If the count remains **0**, PostgreSQL still has TLS enabled on the server, but clients might not yet be connecting with SSL. Check the registry database URI or connection arguments for an **sslmode** value such as **verify-full**, and review Operator Events for certificate or CA errors.

2.1.3. Providing custom TLS certificates

To use your own certificates instead of Operator-generated or Service CA certificates, create a Secret and reference it from the component override.

Prerequisites

- TLS is enabled on the target component (**overrides.tls.enabled: true**).
- You have a Secret containing PEM-encoded **ca.crt**, **tls.crt**, and **tls.key** entries.
- Your TLS certificate includes Subject Alternative Name (SAN) entries that match the PostgreSQL Service DNS names. For the **postgres** component, include the following names:

```
<registry_name>-quay-database
<registry_name>-quay-database.<namespace>
<registry_name>-quay-database.<namespace>.svc
<registry_name>-quay-database.<namespace>.svc.cluster.local
localhost
```

For the **clairpostgres** component, include the following names:

-

```
<registry_name>-clair-postgres
<registry_name>-clair-postgres.<namespace>
<registry_name>-clair-postgres.<namespace>.svc
<registry_name>-clair-postgres.<namespace>.svc.cluster.local
localhost
```

Procedure

1. Create a Secret in the same namespace as your **QuayRegistry** custom resource. For example:

```
apiVersion: v1
kind: Secret
metadata:
  name: <postgres_or_clairpostgres_name>-tls
  namespace: <namespace>
type: Opaque
stringData:
  ca.crt: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----
  tls.crt: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----
  tls.key: |
    -----BEGIN EC PRIVATE KEY-----
    ...
    -----END EC PRIVATE KEY-----
```

2. Reference the Secret from the PostgreSQL component override:

```
    - kind: postgres
      managed: true
      overrides:
        tls:
          enabled: true
          secretRef:
            name: <postgres_or_clairpostgres_name>-tls
```

3. Save the **QuayRegistry** and wait for reconciliation.

Verification

1. Connect to the Red Hat Quay PostgreSQL pod and check that SSL is enabled:

```
$ oc exec -n <namespace> deploy/<registry>-quay-database -c postgres -- bash -lc 'psql -U
"$POSTGRESQL_USER" -d "$POSTGRESQL_DATABASE" -tAc "SHOW ssl;"'
```

   The command should return **on**.

2. Optional: Confirm that client connections use SSL:

```
$ oc exec -n <namespace> deploy/<registry_name>-quay-database -c postgres -- \
```

```
bash -lc 'psql -U "$POSTGRESQL_USER" -d "$POSTGRESQL_DATABASE" -tAc
"SELECT count(*) FROM pg_stat_ssl s JOIN pg_stat_activity a ON s.pid = a.pid WHERE
s.ssl = true AND a.client_addr IS NOT NULL;"'
```

A count greater than **0** indicates active TLS client connections. If the count remains **0**, PostgreSQL still has TLS enabled on the server, but clients might not yet be connecting with SSL. Check the registry database URI or connection arguments for an **sslmode** value such as **verify-full**, and review Operator Events for certificate or CA errors.

2.1.4. Disabling TLS

You can disable TLS on an existing deployment without data loss by removing or setting **overrides.tls.enabled** to **false** on the affected components. The Operator removes TLS configuration from PostgreSQL and restores unencrypted connection strings.

Procedure

1. Edit the **QuayRegistry** custom resource.
2. Set **overrides.tls.enabled: false** or remove the **overrides.tls** block from the **postgres** and/or **clairpostgres** components.
3. Save the changes and wait for the Operator to reconcile the registry.

2.1.5. Troubleshooting Operator-managed PostgreSQL TLS configurations

If TLS configuration is invalid, the Operator sets a **RolloutBlocked** condition on the **QuayRegistry** and emits Kubernetes Events with details.

Common causes include:

- A referenced TLS Secret does not exist
- The Secret is missing **ca.crt**, **tls.crt**, or **tls.key**
- The certificate and private key do not match
- The certificate has expired

On OpenShift Container Platform, if the Service CA serving certificate Secret has not yet been created, the Operator emits Events such as a PostgreSQL service CA error and retries until the Secret is available. A brief **RolloutBlocked** condition during that wait is expected.

If **SHOW ssl;** returns **on** but **pg_stat_ssl** shows no TLS client sessions, PostgreSQL is accepting encrypted connections, but Red Hat Quay or Clair might still be connecting without SSL. Check the registry configuration for an **sslmode** value on the database URI or connection arguments, and review Operator Events for certificate or CA problems.

For external PostgreSQL databases, continue to configure TLS through **DB_URI** and related configuration fields in the config bundle. For more information, see Database configuration fields.

2.2. CONFIGURING CERTIFICATE-BASED AUTHENTICATION WITH SQL

To connect Red Hat Quay to an SQL database with client-side certificates, you can create a **configBundleSecret** that includes TLS certificate files and update **DB_CONNECTION_ARGS** and **DB_URI** in the **config.yaml** file.

This procedure uses CloudSQL as an example and also applies to PostgreSQL and other supported databases.

Prerequisites

- You have generated custom Certificate Authorities (CAs) and your SSL/TLS certificates and keys are available in **PEM** format that will be used to generate an SSL connection with your CloudSQL database. For more information, see SSL and TLS for Red Hat Quay.
- You have **base64 decoded** the original config bundle into a **config.yaml** file. For more information, see Downloading the existing configuration.
- You are using an externally managed PostgreSQL or CloudSQL database. For more information, see Using and existing PostgreSQL database with the **DB_URI** variable set.
- Your externally managed PostgreSQL or CloudSQL database is configured for SSL/TLS.
- The **postgres** component of your **QuayRegistry** CRD is set to **managed: false**, and your CloudSQL database is set with the **DB_URI** configuration variable. The following procedure uses **postgresql://<cloudsql_username>:<dbpassword>@<database_host>:<port>/<database_name>**.

Procedure

1. After you have generated the CAs and SSL/TLS certificates and keys for your CloudSQL database and ensured that they are in **.pem** format, test the SSL connection to your CloudSQL server:

   a. Initiate a connection to your CloudSQL server by entering the following command:

   ```
   $ psql "sslmode=verify-ca sslrootcert=<ssl_server_certificate_authority>.pem sslcert=
   <ssl_client_certificate>.pem sslkey=<ssl_client_key>.pem hostaddr=<database_host>
   port=<5432> user=<cloudsql_username> dbname=<cloudsql_database_name>"
   ```

2. In your Red Hat Quay directory, create a new YAML file, for example, **quay-config-bundle.yaml**, by running the following command:

   ```
   $ touch quay-config-bundle.yaml
   ```

3. Create a **postgresql-client-certs** resource by entering the following command:

   ```
   $ oc -n <quay_namespace> create secret generic postgresql-client-certs \
   --from-file config.yaml=<path/to/config.yaml> \
   --from-file=tls.crt=<path/to/ssl_client_certificate.pem> \
   --from-file=tls.key=<path/to/ssl_client_key.pem> \
   --from-file=ca.crt=<path/to/ssl_server_certificate.pem>
   ```

   where:

   config.yaml=<path/to/config.yaml>

Specifies your base64 decoded **config.yaml** file.

tls.crt=<path/to/ssl_client_certificate.pem>

Specifies your SSL certificate in **.pem** format.

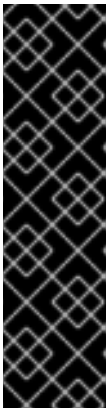
tls.key=<path/to/ssl_client_key.pem>

Specifies your SSL key in **.pem** format.

ca.crt=<path/to/ssl_server_certificate.pem>

Specifies your SSL root CA in **.pem** format.

4. Edit your **`quay-config-bundle.yaml** file to include the following database connection settings:



IMPORTANT

- The information included in the **DB_CONNECTION_ARGS** variable, for example, **sslmode**, **sslrootcert**, **sslcert**, and **sslkey must** match the information appended to the **DB_URI** variable. Failure to match might result in a failed connection.
- You cannot specify custom filenames or paths. Certificate file paths for **sslrootcert**, **sslcert**, and **sslkey** are hardcoded defaults and mounted into the **Quay** pod from the Kubernetes secret. You must adhere to the following naming conventions or it will result in a failed connection.

```
DB_CONNECTION_ARGS:
  autorollback: true
  sslmode: verify-ca
  sslrootcert: /.postgresql/root.crt
  sslcert: /.postgresql/postgresql.crt
  sslkey: /.postgresql/postgresql.key
  threadlocals: true
DB_URI: postgresql://<dbusername>:<dbpassword>@<database_host>:
<port>/<database_name>?sslmode=verify-
full&sslrootcert=/.postgresql/root.crt&sslcert=/.postgresql/postgresql.crt&sslkey=/.postgresql/pos
gresql.key
```

where:

DB_CONNECTION_ARGS.sslmode

Specifies **verify-ca**, which ensures that the database connection uses SSL/TLS and verifies the server certificate against a trusted CA. This can work with both trusted CA and self-signed CA certificates. However, this mode does not verify the hostname of the server. For full hostname and certificate verification, use **verify-full**. For more information about the configuration options available, see PostgreSQL SSL/TLS connection arguments.

DB_CONNECTION_ARGS.sslrootcert

Specifies the **root.crt** file that contains the root certificate used to verify the SSL/TLS connection with your CloudSQL database. This file is mounted in the **Quay** pod from the Kubernetes secret.

DB_CONNECTION_ARGS.sslcert

Specifies the **postgresql.crt** file that contains the client certificate used to authenticate the connection to your CloudSQL database. This file is mounted in the **Quay** pod from the Kubernetes secret.

DB_CONNECTION_ARGS.sslkey

Specifies the **postgresql.key** file that contains the private key associated with the client certificate. This file is mounted in the **Quay** pod from the Kubernetes secret.

DB_CONNECTION_ARGS.threadlocals

Specifies auto-rollback for connections.

DB_URI

Specifies the URI that accesses your CloudSQL database. Must be appended with the **sslmode** type, your **root.crt**, **postgresql.crt**, and **postgresql.key** files. The SSL/TLS information included in **DB_URI** must match the information provided in **DB_CONNECTION_ARGS**. If you are using CloudSQL, you must include your database username and password in this variable.

5. Create the **configBundleSecret** resource by entering the following command:

```
$ oc create -n <namespace> -f quay-config-bundle.yaml
```

Example output

```
secret/quay-config-bundle created
```

6. Update the **QuayRegistry** YAML file to reference the **quay-config-bundle** object by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"configBundleSecret":"quay-config-bundle"}}'
```

Example output

```
quayregistry.quay.redhat.com/example-registry patched
```

7. Ensure that your **QuayRegistry** YAML file has been updated to use the extra CA certificate **configBundleSecret** resource by entering the following command:

```
$ oc get quayregistry <registry_name> -n <namespace> -o yaml
```

Example output

```
# ...
  configBundleSecret: quay-config-bundle
# ...
```

CHAPTER 3. ADDING ADDITIONAL CERTIFICATE AUTHORITIES FOR RED HAT QUAY

Certificate Authorities (CAs) enable Red Hat Quay to verify SSL/TLS connections to external services such as OIDC providers, LDAP servers, and storage backends. The steps for adding additional CAs differ between standalone deployments and Red Hat Quay on OpenShift Container Platform.

3.1. ADDING ADDITIONAL CERTIFICATE AUTHORITIES TO THE RED HAT QUAY CONTAINER

To add Certificate Authorities to a standalone Red Hat Quay container, you can copy CA files into the **extra_ca_certs** directory in your configuration folder and restart the registry container. Red Hat Quay uses these certificates to verify TLS connections to external services.

Prerequisites

- You have a CA for the desired service.

Procedure

1. View the certificate to be added to the container by entering the following command:

```
$ cat storage.crt
```

Example output

```
-----BEGIN CERTIFICATE-----
MIIDTTCCAjWgAwIBAgIJAMVr9ngjJhzbMA0GCSqGSIb3DQEBCwUAMD0xCzAJBgNV...
-----END CERTIFICATE-----
```

2. Create the **extra_ca_certs** in the **/config** folder of your Red Hat Quay directory by entering the following command:

```
$ mkdir -p /path/to/quay_config_folder/extra_ca_certs
```

3. Copy the CA file to the **extra_ca_certs** folder. For example:

```
$ cp storage.crt /path/to/quay_config_folder/extra_ca_certs/
```

4. Ensure that the **storage.crt** file exists within the **extra_ca_certs** folder by entering the following command:

```
$ tree /path/to/quay_config_folder/extra_ca_certs
```

Example output

```
/path/to/quay_config_folder/extra_ca_certs
 ├── storage.crt----
```

5. Obtain the **CONTAINER ID** of your **Quay** consider by entering the following command:

```
$ podman ps
```

Example output

```
CONTAINER ID  IMAGE                          COMMAND               CREATED
STATUS            PORTS
5a3e82c4a75f  <registry>/<repo>/quay:{productminv} "/sbin/my_init"         24 hours ago
Up 18 hours       0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp, 443/tcp   grave_keller
```

6. Restart the container by entering the following command

```
$ podman restart 5a3e82c4a75f
```

7. Confirm that the certificate was copied into the container namespace by running the following command:

```
$ podman exec -it 5a3e82c4a75f cat /etc/ssl/certs/storage.pem
```

Example output

```
-----BEGIN CERTIFICATE-----
MIIDTTCCAjWgAwIBAgIJAMVr9ngjJhzbMA0GCSqGSIb3DQEBCwUAMD0xCzAJBgNV...
-----END CERTIFICATE-----
```

3.2. ADDING ADDITIONAL CERTIFICATE AUTHORITIES TO RED HAT QUAY ON OPENSHIFT CONTAINER PLATFORM

On Red Hat Quay on OpenShift Container Platform, additional Certificate Authorities (CAs) are merged into the registry trust store from the **conf/stack/extra_ca_certs/** directory inside the final configuration bundle. Red Hat Quay uses those CAs to verify TLS to external services such as LDAP, OIDC, and storage endpoints.

This path is the same logical location as the **extra_ca_certs** directory used on standalone deployments. On Red Hat Quay on OpenShift Container Platform, you do not create that directory on a node yourself. Instead, add each CA PEM file to the **configBundleSecret** as its own Secret data entry whose key name begins with **extra_ca_cert_**. The Red Hat Quay Operator extracts these keys when it reconciles the registry and writes the files under **conf/stack/extra_ca_certs/**. The file name in that directory is the key name with the **extra_ca_cert_** prefix removed. For example, a Secret key **extra_ca_cert_ldap-ca.pem** is available at runtime as **conf/stack/extra_ca_certs/ldap-ca.pem**.

You do not add an **extra_ca_certs** list or block to **config.yaml** only to load those CAs on Red Hat Quay on OpenShift Container Platform. The supported approach is the **extra_ca_cert_*** keys in the bundle Secret, as shown in the procedures that follow.

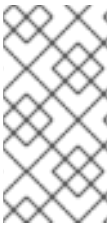
When another setting requires an explicit path to one of those PEM files (for example, **ssl_ca_path** for a given integration), set the path in **config.yaml** to the runtime location under **conf/stack/extra_ca_certs/**, for example:

```
# ...
ssl_ca_path: conf/stack/extra_ca_certs/ldap-ca.pem
# ...
```

The following procedures show you how to download your existing configuration file, include additional CA files in the **configBundleSecret**, and re-apply the bundle so that OpenShift Container Platform deploys the updated configuration.

3.2.1. Modifying the configuration file by using the CLI

To modify the **config.yaml** file for your Red Hat Quay registry and enable new features, you can download the existing configuration from the **configBundleSecret** by using the CLI. After making changes, you can re-upload the **configBundleSecret** resource to apply the changes.



NOTE

Modifying the **config.yaml** file that is stored by the **configBundleSecret** resource is a multi-step procedure that requires base64 decoding the existing configuration file and then uploading the changes. For most cases, using the OpenShift Container Platform web console to make changes to the **config.yaml** file is simpler.

Prerequisites

- You are logged in to the OpenShift Container Platform cluster as a user with admin privileges.

Procedure

1. Describe the **QuayRegistry** resource by entering the following command:

```
$ oc describe quayregistry -n <quay_namespace>
```

```
# ...
  Config Bundle Secret: example-registry-config-bundle-v123x
# ...
```

2. Obtain the secret data by entering the following command:

```
$ oc get secret -n <quay_namespace> <example-registry-config-bundle-v123x> -o
jsonpath='{.data}'
```

```
{
    "config.yaml": "RkVBVFVSRV9VU0 ... MDAwMAo="
}
```

3. Decode the data into a YAML file into the current directory by passing in the **>> config.yaml** flag. For example:

```
$ echo 'RkVBVFVSRV9VU0 ... MDAwMAo=' | base64 --decode >> config.yaml
```

4. Make the desired changes to your **config.yaml** file, and then save the file as **config.yaml**.

5. Create a new **configBundleSecret** YAML by entering the following command.

```
$ touch <new_configBundleSecret_name>.yaml
```

6. Create the new **configBundleSecret** resource, passing in the **config.yaml** file` by entering the following command:

```
$ oc -n <namespace> create secret generic <secret_name> \
  --from-file=config.yaml=</path/to/config.yaml> \
  --dry-run=client -o yaml > <new_configBundleSecret_name>.yaml
```

where:

</path/to/config.yaml>

Specifies your base64 decoded **config.yaml** file.

7. Create the **configBundleSecret** resource by entering the following command:

```
$ oc create -n <namespace> -f <new_configBundleSecret_name>.yaml
```

```
secret/config-bundle created
```

8. Update the **QuayRegistry** YAML file to reference the new **configBundleSecret** object by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"configBundleSecret":"<new_configBundleSecret_name>"}}'
```

```
quayregistry.quay.redhat.com/example-registry patched
```

Verification

1. Verify that the **QuayRegistry** CR has been updated with the new **configBundleSecret**:

```
$ oc describe quayregistry -n <quay_namespace>
```

```
# ...
  Config Bundle Secret: <new_configBundleSecret_name>
# ...
```

After patching the registry, the Red Hat Quay Operator automatically reconciles the changes.

3.2.2. Adding additional Certificate Authorities to Red Hat Quay on OpenShift Container Platform

To add additional Certificate Authorities to Red Hat Quay on OpenShift Container Platform, you can extend the **configBundleSecret** with **extra_ca_cert_*** entries for each CA PEM file and update the **QuayRegistry** custom resource to reference the updated secret. Each key name determines the file name under conf/stack/extra_ca_certs/ at runtime.

Additional CAs are not declared as an **extra_ca_certs** field inside the **config.yaml** file. Each CA is a separate entry in the Secret: the key must start with **extra_ca_cert_**, and the remainder of the key name becomes the file name under **conf/stack/extra_ca_certs/** after the Operator reconciles the registry.

Prerequisites

- You have base64 decoded the original config bundle into a **config.yaml** file. For more information, see Downloading the existing configuration.
- You have a Certificate Authority (CA) file or files.

Procedure

1. Create a new YAML file, for example, **extra-ca-certificate-config-bundle-secret.yaml**:

```
$ touch extra-ca-certificate-config-bundle-secret.yaml
```

2. Create the **extra-ca-certificate-config-bundle-secret** resource.

   a. Create the resource by entering the following command:

```
$ oc -n <namespace> create secret generic extra-ca-certificate-config-bundle-secret \
  --from-file=config.yaml=</path/to/config.yaml> \
  --from-file=extra_ca_cert_<name-of-certificate-one>=<path/to/certificate_one> \
  --from-file=extra_ca_cert_<name-of-certificate-two>=<path/to/certificate_two> \
  --from-file=extra_ca_cert_<name-of-certificate-three>=<path/to/certificate_three> \
  --dry-run=client -o yaml > extra-ca-certificate-config-bundle-secret.yaml
```

   where:

   --from-file=config.yaml=</path/to/config.yaml>

   Specifies your base64 decoded **config.yaml** file.

   --from-file=extra_ca_cert_<name-of-certificate-one>=<path/to/certificate_one>

   Specifies the extra CA file to be added to the system trust bundle.

   --from-file=extra_ca_cert_<name-of-certificate-two>=<path/to/certificate_two>

   Specifies a second CA file to be added into the system trust bundle. This parameter is optional.

   --from-file=extra_ca_cert_<name-of-certificate-three>=<path/to/certificate_three>

   Specifies a third CA file to be added into the system trust bundle. This parameter is optional.

3. Optional. You can check the content of the **extra-ca-certificate-config-bundle-secret.yaml** file by entering the following command:

```
$ cat extra-ca-certificate-config-bundle-secret.yaml
```

Example output

```
apiVersion: v1
data:
  config.yaml: <example_scren>...
  extra_ca_cert_certificate-one: <example_secret>...
  extra_ca_cert_certificate-three: <example_secret>...
  extra_ca_cert_certificate-two: <example_secret>...
kind: Secret
metadata:
```

```
  creationTimestamp: null
  name: extra-ca-certificate-config-bundle-secret
  namespace: <namespace>
```

4. Create the **configBundleSecret** resource by entering the following command:

```
$ oc create -n <namespace> -f extra-ca-certificate-config-bundle-secret.yaml
```

Example output

```
secret/extra-ca-certificate-config-bundle-secret created
```

5. Update the **QuayRegistry** YAML file to reference the **extra-ca-certificate-config-bundle-secret** object by entering the following command:

```
$ oc patch quayregistry <registry_name> -n <namespace> --type=merge -p '{"spec":
{"configBundleSecret":"extra-ca-certificate-config-bundle-secret"}}'
```

Example output

```
quayregistry.quay.redhat.com/example-registry patched
```

6. Ensure that your **QuayRegistry** YAML file has been updated to use the extra CA certificate **configBundleSecret** resource by entering the following command:

```
$ oc get quayregistry <registry_name> -n <namespace> -o yaml
```

Example output

```
# ...
  configBundleSecret: extra-ca-certificate-config-bundle-secret
# ...
```

3.3. ADDING CUSTOM SSL/TLS CERTIFICATES WHEN RED HAT QUAY IS DEPLOYED ON KUBERNETES

To add custom SSL/TLS certificates to your Red Hat Quay deployment on Kubernetes, you can base64 encode the certificate, add it to the config secret, and restart the pods. This procedure works around the limitation where the superuser panel certificate upload function does not work with Kubernetes deployments.

Prerequisites

- Red Hat Quay has been deployed.
- You have a custom **ca.crt** file.

Procedure

1. Base64 encode the contents of an SSL/TLS certificate by entering the following command:

```
$ cat ca.crt | base64 -w 0
```

Example output

```
...c1psWGpqeGlPQmNEWkJPMjJ5d0pDemVnR2QNCnRsbW9JdEF4YnFSdVd3PT0KLS0tLS1
FTkQgQ0VSVElGSUNBVEUtLS0tLQo=
```

2. Enter the following **kubectl** command to edit the **quay-enterprise-config-secret** file:

```
$ kubectl --namespace quay-enterprise edit secret/quay-enterprise-config-secret
```

3. Add an entry for the certificate and paste the full **base64** encoded stringer under the entry. For example:

```
  custom-cert.crt:
c1psWGpqeGlPQmNEWkJPMjJ5d0pDemVnR2QNCnRsbW9JdEF4YnFSdVd3PT0KLS0tLS1F
TkQgQ0VSVElGSUNBVEUtLS0tLQo=
```

4. Use the **kubectl delete** command to remove all Red Hat Quay pods. For example:

```
$ kubectl delete pod quay-operator.v3.7.1-6f9d859bd-p5ftc quayregistry-clair-postgres-
7487f5bd86-xnxpr quayregistry-quay-app-upgrade-xq2v6  quayregistry-quay-database-
859d5445ff-cqthr quayregistry-quay-redis-84f888776f-hhgms
```

Afterwards, the Red Hat Quay deployment automatically schedules replace pods with the new certificate data.